A CTE is a temporary named result set that you define at the beginning of a SQL statement using the `WITH` keyword.

Think of it as creating a temporary, named query that you can reference in your main SQL statement.

CTEs help make complex SQL queries easier to read, understand, and maintain.

1. Basic syntax

```
WITH cte_name AS (
    SELECT column1, column2
    FROM table_name
    WHERE condition
)
SELECT *
FROM cte_name;
```

The CTE exists only for the duration of that SQL statement.

2. Simple example

Suppose we have an `employees` table:

employee	department	salary
Amit	IT	50000
Rahul	IT	70000
Priya	IT	60000

Neha	HR	40000
Ravi	HR	50000

Requirement: Find employees whose salary is greater than the average salary of all employees.

Without CTE:

```
SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Using CTE:

```
WITH avg_salary AS (
    SELECT AVG(salary) AS average
    FROM employees
)
SELECT e.*
FROM employees e
CROSS JOIN avg_salary a
WHERE e.salary > a.average;
```

Here:

1. The CTE calculates the average salary.
2. The main query uses that result.
3. We avoid repeating the average salary calculation in the query.

3. CTE with window functions

This is particularly useful for the window functions you have been learning.

Requirement: Find the top two highest-paid employees in each department.

```
WITH ranked_employees AS (
    SELECT
        employee,
        department,
        salary,
        ROW_NUMBER() OVER (
            PARTITION BY department
            ORDER BY salary DESC
        ) AS rn
    FROM employees
)
SELECT *
FROM ranked_employees
WHERE rn <= 2;
```

How it works:

Step 1: CTE execution

The CTE calculates the row number for each employee.

Employee	Department	Salary	rn
Rahul	IT	70000	1

Priya	IT	60000	2
Amit	IT	50000	3
Ravi	HR	50000	1
Neha	HR	40000	2

Step 2: Main query

`WHERE rn <= 2` filters the CTE result.

Employee	Department	Salary	rn
Rahul	IT	70000	1
Priya	IT	60000	2
Ravi	HR	50000	1
Neha	HR	40000	2

This is a common interview pattern because Oracle does not allow you to filter a window-function result directly in the same query block's `WHERE` clause.

4. Multiple CTEs

You can define multiple CTEs in a single query.

```
WITH department_avg AS (
    SELECT
        department,
        AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
),
high_salary AS (
    SELECT
        e.employee,
        e.department,
        e.salary,
        d.avg_salary
    FROM employees e
    JOIN department_avg d
        ON e.department = d.department
    WHERE e.salary > d.avg_salary
)
SELECT *
FROM high_salary;
```

Execution conceptually:

1. `department_avg` calculates average salary per department.
2. `high_salary` uses the first CTE to find employees above their department average.
3. The final query returns the result.

5. Recursive CTE

A recursive CTE references itself. It is useful for hierarchical data, such as:

- Employee-manager relationships.
- Organization hierarchies.
- Category trees.
- Generating sequences.

Example: Generate numbers from 1 to 5 in Oracle.

```
WITH numbers (n) AS (
   SELECT 1 FROM dual

   UNION ALL

   SELECT n + 1
   FROM numbers
   WHERE n < 5
)
SELECT n
FROM numbers;
```

Result:

N
1
2
3

4

5

The first query is the anchor member. The second query is the recursive member.

6. CTE vs Subquery vs Temporary Table

Feature	CTE	Subquery	Temporary table
Readability	Good	Can become complex	Good
Reusable within statement	Yes	Usually no direct reuse	Yes
Exists beyond statement	No	No	Depends on table definition
Stores data physically	Not necessarily	Not necessarily	Yes
Recursive queries	Yes	Not directly	Requires procedural logic

Important: A CTE is not necessarily materialized or stored in memory as a separate table. Oracle's optimizer may inline it or choose to materialize it depending on the query and execution plan.

7. When should you use a CTE?

Scenario	Why CTE helps

Filtering `ROW_NUMBER()`	Filter window-function results
Multiple calculation steps	Break complex SQL into stages
Hierarchical queries	Recursive traversal
Reusing intermediate results	Reference a named result multiple times
Complex joins and aggregations	Improve readability

My recommendation: Practise these three patterns together:

1. CTE + `ROW_NUMBER()` — Top N employees per department.
2. CTE + `DENSE_RANK()` — Find the second-highest salary.
3. Recursive CTE — Employee-manager hierarchy.

These are particularly useful SQL interview patterns.